

TRABAJO

IMAGEN: SEGUIMIENTO DE JUGADORES CON VISIÓN ARTIFICIAL

Sergio Ripoll Ciges

Carlos Clariana Botella

Marc Forés Domenech

Carlos Molina Castellanos



Índice de contenidos

1. Introducción	2
2. Objetivos del proyecto.....	2
3. Herramientas y tecnologías utilizadas	4
4. Funcionamiento del sistema.....	6
5. Uso de la IA como herramienta de desarrollo.....	11
6. Comparación de técnicas de detección.....	12
7. Resultados obtenidos.....	14
8. Problemas encontrados y cómo los solucionamos.....	16
9. Mejoras futuras	18
10. Conclusión	19
11. Bibliografía.....	20

1. Introducción

El deporte y la tecnología son dos ámbitos que cada vez están más unidos. Si ves un partido de fútbol hoy en día, probablemente hayas visto esos gráficos en pantalla que muestran la velocidad de un jugador, su recorrido durante el partido o el mapa de calor de dónde ha estado más activo. Todo eso no aparece por arte de magia: hay sistemas de visión artificial detrás que analizan el vídeo en tiempo real o casi real para extraer esa información.

La idea de este proyecto viene precisamente de ahí. Nos pareció interesante intentar construir algo parecido, aunque a una escala mucho más modesta y con los medios de los que disponemos como estudiantes. El objetivo era claro desde el principio: que el sistema pudiera coger un vídeo de un partido, detectar a los jugadores, separar los dos equipos de forma automática, hacer un seguimiento de cada uno de ellos a lo largo del partido y, por último, generar mapas de calor tanto en 2D como en 3D para visualizar dónde ha habido más actividad.

Para conseguirlo, hemos usado herramientas modernas de inteligencia artificial, en concreto un modelo de detección de objetos llamado YOLOv8, junto con OpenCV para el procesamiento de vídeo y varias librerías adicionales para el seguimiento y la visualización. Además, hemos apoyado el desarrollo en herramientas de IA generativa, que nos han ayudado a programar y resolver dudas a lo largo del proceso, aunque las decisiones de diseño, los objetivos y la lógica del proyecto son completamente nuestros.

La visión artificial, de forma simplificada, es la rama de la informática que se encarga de enseñar a los ordenadores a "ver" e interpretar imágenes y vídeos. Gracias al deep learning, es decir, al uso de redes neuronales profundas entrenadas con millones de imágenes, hoy en día se pueden detectar objetos, personas o acciones en un vídeo.

2. Objetivos del proyecto

El objetivo principal del proyecto era construir un sistema que, dado un vídeo de un partido de fútbol, fuese capaz de realizar automáticamente las siguientes tareas:

- **Detectar a los jugadores** en cada fotograma del vídeo, diferenciando entre jugadores, árbitros y otros elementos del campo.
- **Separar los equipos** de forma automática según el color de sus camisetas, sin tener que decirle al sistema manualmente qué equipo es cuál.
- **Hacer seguimiento de cada jugador** a lo largo del vídeo, de modo que el sistema mantenga la identidad de cada persona, aunque se mueva por el campo, se tape con otro jugador o salga brevemente de cámara.
- **Detectar el balón** y seguir su trayectoria en la medida de lo posible.
- **Generar un mapa de calor 2D** que muestre en una vista aérea del campo las zonas donde cada equipo ha tenido más presencia a lo largo del partido.
- **Generar un mapa de calor 3D** que represente la misma información de forma tridimensional, añadiendo una dimensión visual extra que permite ver la densidad de actividad de otra manera.

Todos estos objetivos se plantearon desde el principio del proyecto y han guiado todas las decisiones que hemos tomado durante el desarrollo. No todos han sido igual de sencillos de conseguir, y algunos nos han dado más de un dolor de cabeza, como explicamos más adelante.

3. Herramientas y tecnologías utilizadas

Para desarrollar el proyecto hemos usado varias herramientas y librerías de Python. No vamos a entrar en detalles técnicos muy profundos sobre cómo funcionan internamente, pero sí explicamos para qué sirve cada una y por qué la elegimos.

Python

El lenguaje de programación con el que está hecho todo el proyecto. Es el estándar en proyectos de inteligencia y visión artificiales, tiene una comunidad enorme y una cantidad brutal de librerías disponibles.

YOLOv8 (Ultralytics)

Es el modelo de detección de objetos que usamos para detectar a los jugadores y el balón en cada fotograma. YOLO son las siglas de You Only Look Once, y la idea es que es muy rápido porque analiza la imagen entera de una sola vez en lugar de ir trozo a trozo. La versión 8 es bastante moderna y da muy buenos resultados incluso sin necesidad de entrenarla de cero para fútbol.

OpenCV

Es la librería que usamos para todo lo relacionado con el vídeo: leerlo fotograma a fotograma, dibujar las anotaciones encima y guardar el resultado final. Es la herramienta estándar para este tipo de tareas.

Supervision (Roboflow)

Una librería que facilita muchísimo trabajar con detecciones. En lugar de tener que programar desde cero cómo dibujar las cajas, gestionar el seguimiento o calcular qué detección corresponde a qué jugador, Supervision ofrece funciones ya hechas para todo eso.

ByteTrack

Es el algoritmo de seguimiento que usamos para mantener la identidad de cada jugador entre fotogramas.

Viene integrado en Supervision y nos ha funcionado bastante bien.

Matplotlib / NumPy

Las usamos para generar los mapas de calor 2D y 3D a partir de las posiciones acumuladas de los jugadores durante todo el vídeo.

Resumen de tecnologías

Herramienta	Para qué la usamos
Python 3.10+	Lenguaje principal del proyecto
YOLOv8	Detección de jugadores y balón en cada fotograma
OpenCV	Lectura, anotación y escritura del vídeo
Supervision	Gestión del seguimiento y anotaciones visuales
ByteTrack	Seguimiento multibjeto entre fotogramas

Herramienta	Para qué la usamos
Matplotlib	Generación de mapas de calor 2D y 3D
NumPy	Cálculos y matrices de posición acumulada
CUDA (GPU)	Aceleración del procesamiento (si hay tarjeta NVIDIA)

4. Funcionamiento del sistema

El sistema funciona como una cadena de pasos que se ejecutan para cada fotograma del vídeo. Aquí explicamos qué pasa en cada etapa:

4.1. Lectura del vídeo

Lo primero que hace el programa es abrir el vídeo de entrada y leer sus características: resolución, fotogramas por segundo (FPS) y duración total. A partir de ahí, va leyendo el vídeo fotograma a fotograma en un bucle, procesando cada imagen antes de pasar a la siguiente.

4.2. Detección de jugadores y balón

Cada fotograma se pasa al modelo YOLOv8, que analiza la imagen y devuelve una lista de detecciones.

Cada detección incluye: dónde está el objeto (las coordenadas del rectángulo que lo rodea), qué clase es (persona o balón) y con qué nivel de confianza lo ha identificado. Nosotros filtramos solo las clases que nos interesan y descartamos las detecciones con una confianza muy baja para reducir los errores.

El modelo viene preentrenado con un dataset enorme llamado COCO, que incluye la clase 'persona' y 'balón de deporte', por lo que no hemos necesitado entrenarlo desde cero.

4.3. Clasificación por equipos

Una de las partes más interesantes del proyecto fue conseguir que el sistema separase automáticamente los dos equipos. Para hacerlo, usamos el color de las camisetas: recortamos la parte superior del bounding box de cada jugador (donde está la camiseta) y analizamos su color dominante en el espacio de color HSV.

Con las detecciones de los primeros fotogramas, aplicamos un algoritmo de clustering (K-Means con $k=2$) sobre los colores dominantes de todos los jugadores para distinguir automáticamente los dos grupos de camisetas. A partir de ahí, cada jugador nuevo se asigna al equipo cuyo color sea más parecido al suyo. El resultado es que en el vídeo de salida cada equipo aparece con un color de caja diferente, lo que hace muy fácil distinguirlos visualmente.

4.4. Seguimiento entre fotogramas

La detección sola no es suficiente: si cada fotograma se analizase de forma independiente, no podríamos saber si el jugador número 7 de este fotograma es el mismo que el número 7 del fotograma anterior. El seguimiento (tracking) soluciona esto.

El algoritmo ByteTrack asigna un identificador único a cada jugador y lo mantiene, aunque haya breves pérdidas de detección, por ejemplo, cuando un jugador queda tapado por otro. Para ello, predice dónde debería estar el jugador en el siguiente fotograma y compara esa predicción con las nuevas detecciones para decidir cuál corresponde a quién.

4.5. Mapas de calor 2D

Para generar el mapa de calor 2D, acumulamos la posición central de cada jugador en todos los fotogramas del vídeo. El resultado es una matriz donde cada celda representa una zona del campo y su valor indica cuántas veces ha habido un jugador en esa zona.

Esa matriz se visualiza como un mapa de calor usando Matplotlib, con una escala de colores que va del azul (poca actividad) al rojo (mucha actividad). Lo hacemos por separado para cada equipo y también una versión combinada. La vista es cenital, como si mirásemos el campo desde arriba.

4.6. Mapas de calor 3D

La versión 3D del mapa de calor usa la misma matriz de posiciones, pero la representa con una superficie tridimensional donde el eje Z indica la densidad de actividad en cada zona. Se puede rotar e interactuar con él, lo que da una perspectiva diferente y bastante visual de dónde se ha concentrado el juego.

Para generarlo usamos la función `plot_surface` de Matplotlib con un mapa de colores degradado. No es una visualización interactiva en el vídeo final, sino que se exporta como imagen o figura aparte.

4.7. Exportación del vídeo anotado

Al final del procesamiento, el programa genera un vídeo de salida en el que cada fotograma tiene dibujadas encima las cajas de detección de cada jugador con el color de su equipo, su identificador de seguimiento y la etiqueta de la clase. El vídeo se guarda en formato MP4 con la misma resolución y FPS que el original.

4.8. Fragmentos de código representativos

A continuación, mostramos algunos fragmentos del código que ilustran las partes más importantes del sistema. No es el código completo, sino los bloques más representativos:

Inicialización y carga del modelo:

```
from ultralytics import YOLO
import supervision as sv
import cv2
```

```
# Cargamos el modelo preentrenado YOLOv8 small
modelo = YOLO('yolov8s.pt')
```

```
# Abrimos el video de entrada
cap = cv2.VideoCapture('partido.mp4')
fps = cap.get(cv2.CAP_PROP_FPS)
ancho = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
alto = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
```

Detección y seguimiento en el bucle principal:

```
tracker = sv.ByteTrack()  
box_ann = sv.BoundingBoxAnnotator(thickness=2)  
lbl_ann = sv.LabelAnnotator(text_scale=0.5)
```

```
while cap.isOpened():  
    ret, frame = cap.read()  
    if not ret: break
```

```
# Inferencia: solo clases persona (0) y balon (32)  
resultado = modelo(frame, classes=[0, 32], conf=0.35)[0]  
detecciones = sv.Detections.from_ultralytics(resultado)
```

```
# Actualizamos el tracker con las nuevas detecciones  
detecciones = tracker.update_with_detections(detecciones)
```

```
# Determinamos el equipo por color de camiseta  
colores = clasificar_equipos(frame, detecciones)
```

```
# Anotamos y guardamos el frame  
frame = box_ann.annotate(frame, detecciones)  
out.write(frame)
```

Generación del mapa de calor:

```
import numpy as np
import matplotlib.pyplot as plt
```

```
# posiciones_equipo1 es una lista de (x, y) de todos los frames
heatmap, xedges, yedges = np.histogram2d(
[p[0] for p in posiciones_equipo1],
[p[1] for p in posiciones_equipo1],
bins=50
)
```

```
# Mapa 2D
plt.imshow(heatmap.T, origin='lower', cmap='hot')
plt.colorbar(label='Actividad')
plt.title('Mapa de calor - Equipo 1')
plt.savefig('heatmap_equipo1_2d.png')
```

```
# Mapa 3D
fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')
X, Y = np.meshgrid(xedges[:-1], yedges[:-1])
ax.plot_surface(X, Y, heatmap.T, cmap='hot')
plt.savefig('heatmap_equipo1_3d.png')
```

5. Uso de la IA como herramienta de desarrollo

Queremos ser transparentes en este punto porque forma parte del contexto real del proyecto: durante el desarrollo hemos utilizado herramientas de inteligencia artificial generativa como apoyo puntual mientras programábamos.

Principalmente las hemos usado para ayudarnos a resolver errores, entender algunos problemas concretos del código y recibir sugerencias o recomendaciones de mejora durante el desarrollo. En algunos momentos también nos sirvieron para orientarnos cuando una idea no funcionaba como esperábamos o cuando queríamos buscar una forma más eficiente de implementar alguna parte del proyecto.

Aun así, el planteamiento del proyecto, los objetivos y las decisiones importantes las hemos tomado nosotros. Decidimos qué queríamos conseguir, cómo organizar el sistema y qué funcionalidades implementar, como el seguimiento de jugadores, la separación por equipos o la generación de mapas de calor en 2D y 3D.

Por tanto, la IA ha sido simplemente una herramienta de apoyo durante el desarrollo, similar a consultar documentación, foros o recursos online, pero de una forma más rápida e interactiva.

6. Comparación de técnicas de detección

Antes de decidir usar YOLOv8, miramos qué otras opciones existían para la detección de objetos. Aquí explicamos brevemente las principales alternativas y por qué al final nos quedamos con YOLO.

6.1. Tabla comparativa

Técnica	Velocidad	Precisión	Dificultad de uso	¿Sirve para tiempo real?
YOLOv8	Muy alta	Muy alta	Media	Sí
Faster R-CNN	Baja	Muy alta	Alta	No (muy lento)
SSD MobileNet	Alta	Media	Media	Sí, pero peor precision
Haar Cascades	Muy alta	Baja	Baja	Sí, pero muchos errores
DETR	Media	Alta	Alta	Con dificultades

Haar Cascades

Es una técnica bastante antigua (de principios de los 2000) que sigue funcionando para cosas sencillas como detectar caras de frente. El problema es que para detectar personas en movimiento en un partido de fútbol falla mucho: genera muchos falsos positivos, le afecta mucho la iluminación y no distingue bien entre personas y otros objetos similares. No era una opción viable para nuestro caso.

SSD (Single Shot MultiBox Detector)

Es una alternativa más moderna y rápida que los detectores de dos etapas. Con MobileNet como backbone es bastante ligera y puede funcionar en tiempo real, pero en términos de precisión se queda por detrás de YOLOv8, especialmente para objetos pequeños como el balón.

Faster R-CNN

Es posiblemente el detector más preciso de los que miramos, pero tiene un problema gordo: es lento. Procesa unos 5-15 fotogramas por segundo en GPU, lo que para un vídeo de fútbol a 25-30 FPS no es suficiente. Sería útil si solo quisiéramos analizar imágenes estáticas, pero no para vídeo en tiempo real.

YOLOv8: por qué lo elegimos

YOLOv8 nos da el mejor equilibrio entre velocidad y precisión. En GPU procesa cómodamente por encima de los 30 FPS que necesitamos, detecta bien tanto personas como balones, y su API es muy fácil de usar. Además, la integración con Supervision y ByteTrack está muy bien documentada, lo que nos facilitó mucho el trabajo.

7. Resultados obtenidos

Probamos el sistema con un fragmento de unos 5 minutos de un partido de fútbol grabado en resolución 1080p a 25 FPS. El ordenador que usamos tiene una GPU NVIDIA RTX 3060 con 12 GB de VRAM, que es lo que teníamos disponible.

7.1. Rendimiento

Configuración	FPS promedio	Tiempo total de proceso
Con GPU (RTX 3060)	~47 FPS	Aprox. 6 min para 5 min de vídeo
Sin GPU (solo CPU)	~10 FPS	Aprox. 30 min para 5 min de vídeo

Con GPU el procesamiento es muy fluido y bastante más rápido que el vídeo original. Sin GPU el sistema funciona pero es lento, algo esperado porque estas redes neuronales están pensadas para ejecutarse en GPU.

7.2. Calidad de la detección

Elemento detectado	Precisión aproximada	Observaciones
Jugadores	~90%	Funciona bien en la mayoría de situaciones
Árbitros	~85%	Se detectan como personas, sin distinción propia
Balón	~70%	Falla en tiros rápidos y planos muy abiertos

La detección de jugadores nos ha dado buenos resultados en general. El punto más flojo ha sido el balón, que por su tamaño pequeño y la velocidad que puede llevar, a veces no se detecta bien o se detecta en el sitio equivocado.

7.3. Clasificación por equipos

La separación automática por equipos ha funcionado razonablemente bien cuando los colores de las camisetas son suficientemente distintos. En el vídeo de prueba, con un equipo en camiseta roja y otro en azul, el sistema los diferenciaba correctamente en más del 85% de los casos. El mayor problema aparecía cuando el árbitro, vestido de negro, interfería con el clustering o cuando había jugadores muy juntos.

7.4. Mapas de calor

Los mapas de calor son quizás la parte del proyecto de la que estamos más contentos. El resultado visual es muy claro: se puede ver fácilmente dónde ha dominado cada equipo, qué zonas del campo han estado más activas y cómo se ha distribuido el juego. La versión 3D añade un punto visual interesante que da una perspectiva diferente de la intensidad del juego en cada zona.

8. Problemas encontrados y cómo los solucionamos

El desarrollo no fue en línea recta. Hubo bastantes problemas por el camino, algunos más fáciles de resolver y otros que nos costaron varios días. Aquí los más importantes:

El tracker perdía la identidad de los jugadores cuando se juntaban

Cuando dos jugadores se cruzan o están muy juntos, el algoritmo de seguimiento a veces se confunde y les intercambia los identificadores. Esto es un problema conocido en el campo del tracking multibjeto y no tiene una solución perfecta sin métodos más avanzados. Lo que hicimos fue ajustar los parámetros del tracker (el umbral de asociación y el buffer de frames perdidos) para reducir la frecuencia de estos errores, aunque no desaparecen del todo.

El balón se detectaba mal o no se detectaba

El balón es pequeño, va rápido y a veces está oculto por jugadores. En planos abiertos del campo apenas se ve en la imagen. YOLOv8 preentrenado en COCO no está optimizado especialmente para balones de fútbol, así que la detección fue irregular. Lo paliamos bajando el umbral de confianza mínima para la clase balón, pero eso también aumenta los falsos positivos. Es una de las limitaciones del proyecto que señalamos para mejorar.

El sistema detectaba espectadores en las gradas como jugadores

El modelo detecta cualquier persona, no solo las que están en el campo. Al principio el vídeo estaba lleno de cajas sobre el público de las gradas. Lo solucionamos definiendo una región de interés (ROI) que limita el área de detección al terreno de juego, ignorando todo lo que esté fuera de ese rectángulo.

La clasificación por equipos fallaba con árbitros y porteros

Los árbitros (normalmente de negro) y los porteros (con camiseta diferente al resto del equipo) rompían el clustering de K-Means. Para reducir el impacto, filtramos manualmente el color negro del clustering y añadimos una tercera categoría para detectores de outliers.

Rendimiento en CPU muy lento

Sin GPU, el sistema tardaba demasiado. No encontramos una solución perfecta para esto, pero implementamos un modo de menor resolución de entrada al modelo que mejora la velocidad a costa de algo de precisión.

9. Mejoras futuras

El sistema actual cumple los objetivos que nos pusimos al principio, pero hay cosas que podrían mejorarse bastante si se dedicara más tiempo al proyecto:

- **Entrenar el modelo específicamente para fútbol:** Si entrenásemos YOLOv8 con un dataset de imágenes etiquetadas específicamente de partidos de fútbol, la detección del balón y de los jugadores mejoraría bastante. Existen datasets públicos como SoccerNet que se podrían usar para esto.
- **Mejor re-identificación de jugadores:** Usar algoritmos más avanzados como DeepSORT o BoTSORT, que mantienen una representación visual de cada jugador y pueden re-identificarlos incluso después de que hayan estado fuera de cámara varios segundos.
- **Análisis táctico automático:** Si proyectamos las posiciones de los jugadores sobre un modelo 2D del campo real (con las medidas en metros), se podrían calcular automáticamente la formación de cada equipo, la distancia recorrida por cada jugador o el tiempo de posesión.
- **Interfaz gráfica:** Actualmente el sistema se ejecuta por línea de comandos. Una interfaz gráfica sencilla donde puedas arrastrar el vídeo y ver los resultados en pantalla haría el sistema mucho más usable para alguien sin conocimientos técnicos.
- **Mapa de calor en tiempo real:** En lugar de generarlo al final, mostrarlo superpuesto sobre el campo mientras se procesa el vídeo.
- **Soporte para múltiples cámaras:** En un estadio real hay varias cámaras. Combinar la información de todas ellas daría un seguimiento mucho más completo y sin puntos ciegos.

10. Conclusión

Estamos contentos con el resultado del proyecto. Al principio la idea de hacer un sistema que detectase jugadores los separase por equipos y generase mapas de calor nos parecía bastante ambiciosa, y ha resultado serlo. Hemos tenido que aprender a usar herramientas que no conocíamos, entender cómo funciona el tracking multibjeto, lidiar con problemas que no esperábamos y tomar decisiones técnicas que no siempre eran obvias.

El resultado final hace lo que queríamos: coges un vídeo de un partido, lo procesas con el sistema y obtienes un vídeo anotado con los jugadores identificados y separados por equipos, más los mapas de calor 2D y 3D. No es perfecto, hay cosas que fallan y limitaciones claras que hemos documentado, pero como proyecto de asignatura cumple con creces lo que nos propusimos.

También queremos destacar el aprendizaje que ha supuesto trabajar con IA como herramienta de desarrollo. Hemos aprendido que la IA es muy útil para acelerar ciertos pasos, especialmente cuando tienes claro qué quieres hacer, pero no sabes exactamente cómo hacerlo en código. Pero también hemos comprobado que sin entender lo que estás haciendo es imposible corregir errores o tomar buenas decisiones de diseño.

En definitiva, ha sido un proyecto que nos ha enseñado mucho, tanto técnicamente como en términos de metodología de trabajo, y que nos ha dado una visión mucho más real de cómo funciona la visión artificial aplicada a algo concreto.

11. Bibliografía

- [1] Jocher, G., Chaurasia, A., & Qiu, J. (2023). *Ultralytics YOLOv8*. GitHub.
<https://github.com/ultralytics/ultralytics>
- [2] Zhang, Y. et al. (2022). *ByteTrack: Multi-Object Tracking by Associating Every Detection Box*. Proceedings of ECCV 2022. Springer.
- [3] Bradski, G. (2000). *The OpenCV Library*. Dr. Dobb's Journal of Software Tools, 25, 120-125.
- [4] Roboflow (2023). *Supervision: Computer Vision Toolbox*. GitHub.
<https://github.com/roboflow/supervision>
- [5] Lin, T. Y. et al. (2014). *Microsoft COCO: Common Objects in Context*. ECCV 2014. Springer, Cham.
- [5] Bewley, A. et al. (2016). *Simple Online and Realtime Tracking (SORT)*. Proceedings of ICIP 2016. IEEE.
- [6] Szeliski, R. (2022). *Computer Vision: Algorithms and Applications* (2.a ed.). Springer.
- [7] Documentación oficial de Matplotlib: <https://matplotlib.org/stable/>
- [8] Documentación oficial de NumPy: <https://numpy.org/doc/>
- [9] SoccerNet — Dataset de análisis de fútbol: <https://www.soccer-net.org/>